

Aho-Corasick Automaton Optimization C++17 Class Implementation

Presenter: Dmytro Yuzkevych
Faculty of Mathematics and Computer Science
Jagiellonian University

1. Task & Constraints

- **Input:**

- A text stream **T** (length n).
- A dictionary of words **P** (total length m , count k).

- **Goal:** Find all matches of all words at the same time.

- **Rules:**

- Read text character by character (only forward).
- No backtracking (cannot move the text pointer back).
- Time per character must be constant: $O(1)$.

2. Why Simple Approaches Fail

- **Running KMP for each word:**
 - Time complexity becomes $O(k \cdot n + m)$.
 - If we have thousands of words (k), it works too slowly.
- **Moving pointer back:**
 - Naive algorithms need to return back in text when a mismatch happens.
 - This is impossible if we read data from a live network or a stream.

3. Code Design: Classes vs. Global Arrays

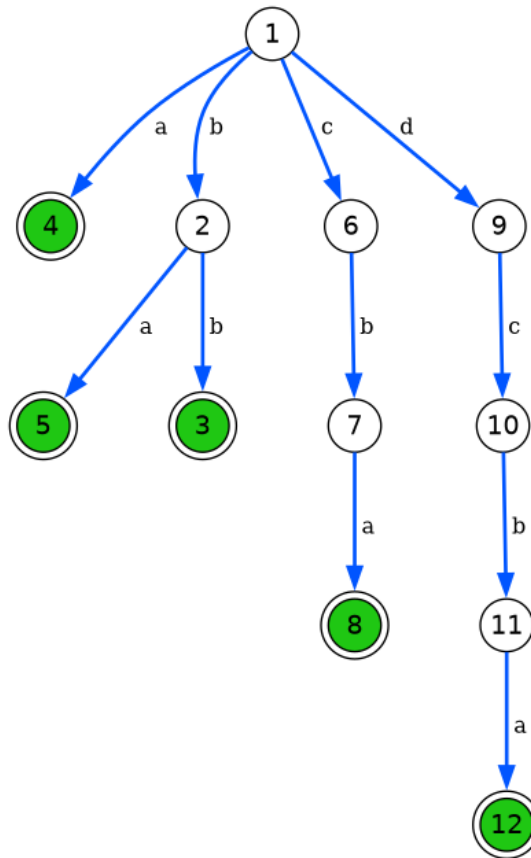
In competitive programming (CP), people often use one global matrix: `int go[MAX_STATES][256]`.

This project uses a clean Object-Oriented approach:

- **No hardcoded limits:** Memory allocates dynamically.
- **Safe memory:** Nodes are managed by `std::unique_ptr<TrieNode>`. When the `Trie` object dies, all memory is deleted automatically (RAII).
- **Isolation:** You can create multiple independent `Trie` objects in your program.

4. Visualizing a Prefix Tree (Trie)

Baseline patterns: "a", "ba", "bba", "abba", "cba", "dcba".



5. Node Structure: TrieNode

Each state in our automaton is defined by a `TrieNode` structure. It stores pointers for tree links, failure shortcuts, and dictionary paths:

```
struct TrieNode {  
    u_ptr<TrieNode> children[SIZE_OF_ALPHABET] = {[0]=nullptr};  
    TrieNode *fail_link = nullptr;  
    TrieNode *dict_link = nullptr;  
    TrieNode *go[SIZE_OF_ALPHABET] = {[0]=nullptr};  
    bool end_of_word = false;  
    int idx_of_word = -1;  
    int id = -1;  
};
```

6. Main Class: Trie Interface

The `Trie` class encapsulates the root nodes and manages operations for building the complete automation machine:

```
class Trie {
private:
    vector<string> all_words;
    u_ptr<TrieNode> Root;
    int number_of_nodes = 1;
    void build_failed_and_dict_links();
    void upgrade_failed_links();

public:
    Trie() : Root(std::make_unique<TrieNode>()) { Root.get()->id = 1; }
    Trie(std::vector<std::string> patterns);
    void insert(string word);

    void build();
    vector<std::pair<int, int>> find_occurrences(string text);
    void make_dot_file(string FileName);
};
```

7. Tree Construction: Insert Method

Standard insertion into the Trie buffer before precalculating failure transitions:

```
inline void Trie::insert(string word) {
    TrieNode *cur = Root.get();
    for (unsigned char c : word) {
        if (cur->children[c] == nullptr) {
            cur->children[c] = std::make_unique<TrieNode>();
        }
        cur = cur->children[c].get();
        if (cur->id == -1) {
            number_of_nodes++;
            cur->id = number_of_nodes;
        }
    }

    cur->end_of_word = true;
    cur->idx_of_word = all_words.size();
    all_words.push_back(x: word);
}
```


8. BFS Preprocessing: Root Initialization

We initialize the queue and set up flat transitions for all direct children of the Root node:

```
Root->fail_link = Root.get();
queue<TrieNode *> q;
for (int i = 0; i < SIZE_OF_ALPHABET; ++i) {
    if (Root->children[i]) {
        Root->children[i]->fail_link = Root.get();
        q.push(x: Root->children[i].get());
    }
}
```

9. BFS Preprocessing: Link Propagation

The main traversal loop propagates standard failure links downwards and compresses suffix paths:

```
while (!q.empty()) {
    TrieNode *cur = q.front();
    q.pop();
    for (int i = 0; i < SIZE_OF_ALPHABET; ++i) {
        TrieNode *child = cur->children[i].get();
        if (child) {
            q.push(x: child);

            TrieNode *fail_cur = cur->fail_link;

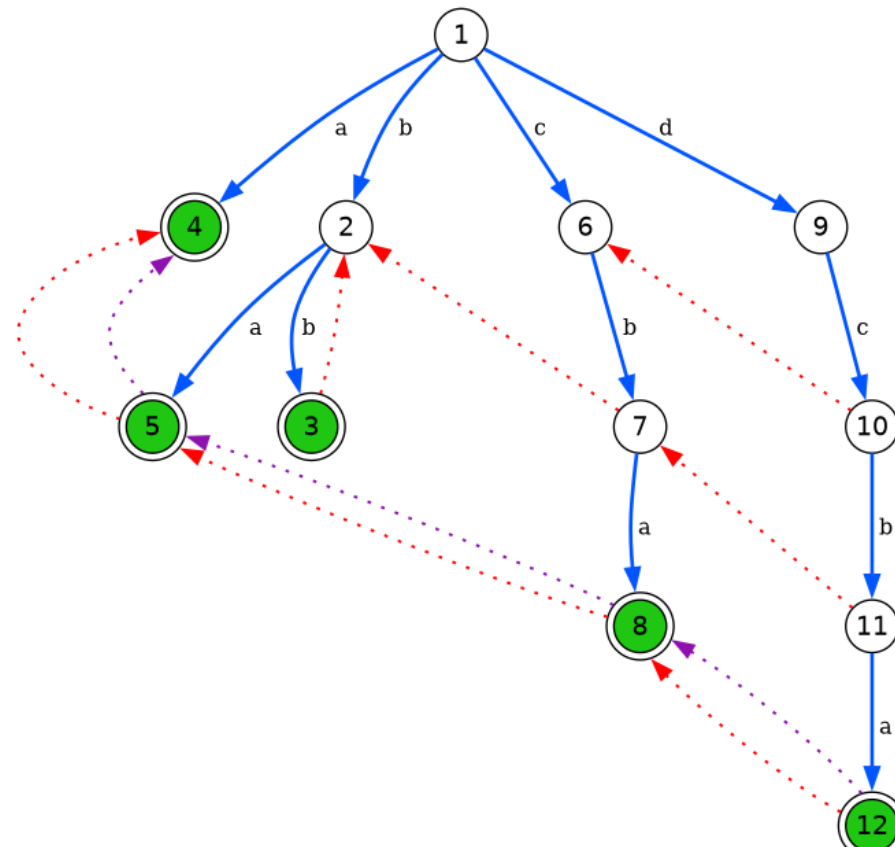
            while (fail_cur != Root.get() && fail_cur->children[i] == nullptr) {
                fail_cur = fail_cur->fail_link;
            }

            if (fail_cur->children[i] != nullptr &&
                fail_cur->children[i].get() != child) {
                child->fail_link = fail_cur->children[i].get();
            } else {
                child->fail_link = Root.get();
            }

            if (child->fail_link->end_of_word) {
                child->dict_link = child->fail_link;
            } else {
                child->dict_link = child->fail_link->dict_link;
            }
        }
    }
}
```

9a. Visualizing Suffix & Dictionary Links

The resulting structural graph after running `build_failed_and_dict_links()`. Suffix paths cross different branches of the tree:



10. Naive Pattern Matching & "Link-Climbing"

```
inline vector<std::pair<int, int>> Trie::find_occurrences(string text) {
    TrieNode *cur = Root.get();
    vector<std::pair<int, int>> answer;
    for (int i = 0; i < text.size(); ++i) {
        unsigned char c = text[i];

        while (cur != Root.get() && cur->children[c] == nullptr) {
            cur = cur->fail_link;
        }

        if (cur->children[c] != nullptr) {
            cur = cur->children[c].get();
        } else {
            cur = Root.get();
        }

        if (cur->end_of_word) {
            answer.push_back(x: {x: i - all_words[cur->idx_of_word].size() + 1, &y: i});
        }

        TrieNode *tempNode = cur->dict_link;

        while (tempNode != nullptr) {
            answer.push_back(x: {x: i - all_words[tempNode->idx_of_word].size() + 1, &y: i});
            tempNode = tempNode->dict_link;
        }
    }
    return answer;
}
```

10a. The Bottleneck: Iterative State Fallback

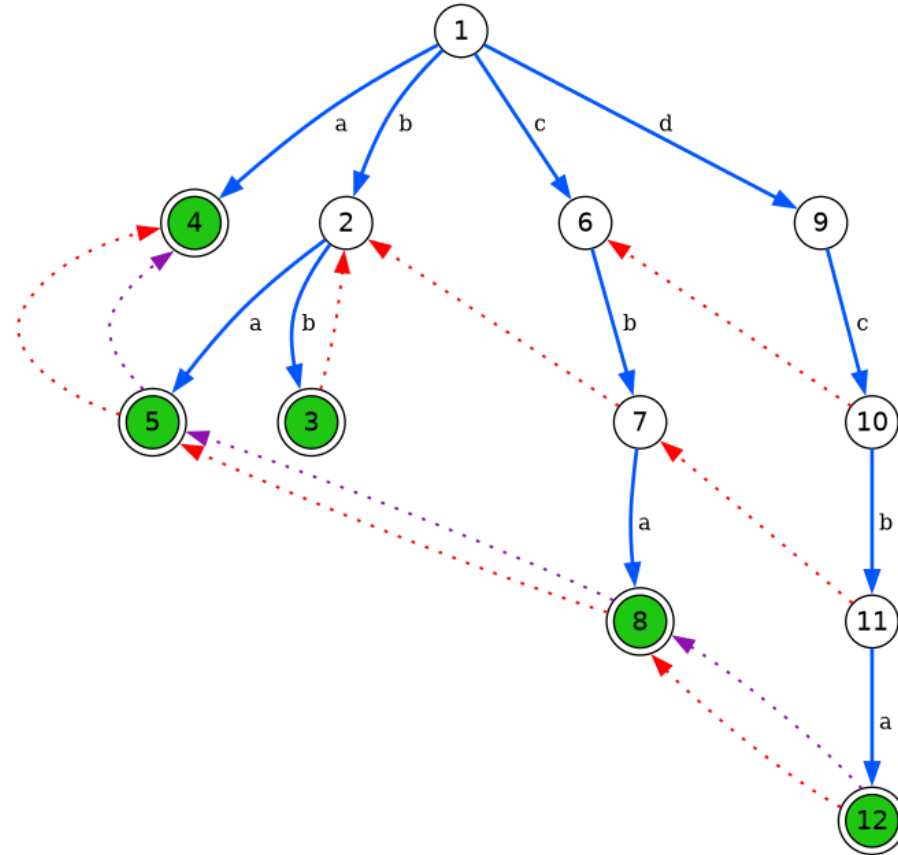
When feeding text like "dcba", reading a mismatching character at the deep node **12** triggers an iterative pointer traversal across branches:

- **Worst-Case Evaluation:** From node **12** ("dcba"), a mismatch forces successive fallback hops through the red dotted suffix links:

Node 12 $\rightarrow$ 8 $\rightarrow$ 5 $\rightarrow$ 4 $\rightarrow$ 1 (Root)

- **Runtime Cost:** Multiple random memory lookups take place during a single text character step.

10b. Link-Climbing Visualization



11. Optimization: DP State Flattening

To eliminate runtime fallback loops, we transform the fallback paths into a flat, complete routing table using **Dynamic Programming**:

$$\text{state} \rightarrow go[c] = \begin{cases} \text{state} \rightarrow \text{children}[c].\text{get}() & \text{if explicit edge exists} \\ \text{state} \rightarrow \text{fail_link} \rightarrow go[c] & \text{otherwise} \end{cases}$$

- **Object-Oriented Design:** Instead of a global 2D matrix, each individual `TrieNode` manages its own flat array of 256 state transitions.
- **Result:** Every fallback route is calculated during the preprocessing stage, resulting in a Complete DFA.

12. Preprocessing Phase 2: DP State Flattening

The `upgrade_failed_links()` method builds the final routing table, removing runtime fallback paths:

```
inline void Trie::upgrade_failed_links() {
    queue<TrieNode*> q;
    for (int i = 0; i < SIZE_OF_ALPHABET; ++i) {
        if (Root->children[i]) {
            q.push(x: Root->children[i].get());
            Root->go[i] = Root->children[i].get();
        } else {
            Root->go[i] = Root.get();
        }
    }

    while (!q.empty()) {
        TrieNode *cur = q.front();
        q.pop();

        for (int i = 0; i < SIZE_OF_ALPHABET; ++i) {
            if (cur->children[i]) {
                q.push(x: cur->children[i].get());
                cur->go[i] = cur->children[i].get();
                continue;
            }
            cur->go[i] = cur->fail_link->go[i];
        }
    }
}
```


13. Optimized Real-Time Search Method

Now, text processing requires exactly **one** table lookup per character. Suffix matching runs efficiently via compressed dictionary shortcuts:

```
inline vector<std::pair<int, int>> Trie::find_occurrences(string text) {
    TrieNode *cur = Root.get();
    vector<std::pair<int, int>> answer;
    for (int i = 0; i < text.size(); ++i) {
        unsigned char c = text[i];
        cur = cur->go[c];

        if (cur->end_of_word) {
            answer.push_back(x: {x: i - all_words[cur->idx_of_word].size() + 1, &y: i});
        }

        TrieNode *tempNode = cur->dict_link;

        while (tempNode != nullptr) {
            answer.push_back(x: {x: i - all_words[tempNode->idx_of_word].size() + 1, &y: i});
            tempNode = tempNode->dict_link;
        }
    }
    return answer;
}
```

14. Complexities & Summary

By integrating DP flattening into our object-oriented Trie design, we achieve optimal performance:

- **Automaton Preprocessing:** $O(m \cdot \Sigma)$ time and memory (where m is total pattern length, $\Sigma = 256$).
- **Stream Text Search:** Strictly $O(n + \text{matches})$ processing bounds.
- **Deterministic Runtime:** Guaranteed $O(1)$ state navigation step per symbol, ideal for live network data streams.